

OPERATING SYSTEMS LAB

System Call Experiments

Detail	Information
Name	Aayush
Roll NO.	2401010042
Program	B.Tech Computer Science and Engineering
Section	B

Index of Experiments

No.	Experiment	System call(s)
1	Process ID and Parent Process ID	getpid(), getppid()
2	Creating a process	fork()
3	Parent waiting for child	wait()
4	Writing to a file	open(), write(), close()
5	Reading from a file	open(), read(), close()
6	Replacing a process image	exec()
7	Complete process creation	fork(), wait(), getpid()

Experiment 1: getpid() and getppid()

Aim

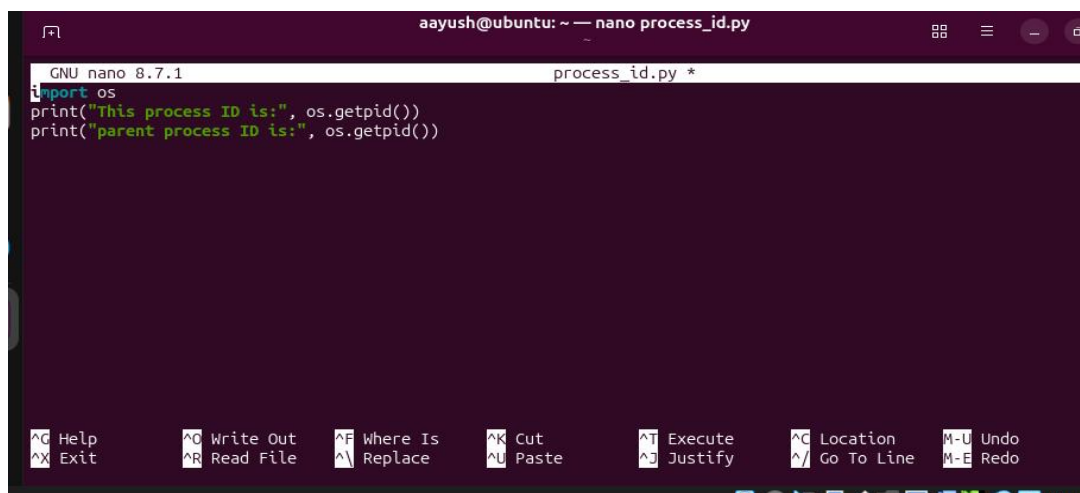
To demonstrate how a process can obtain its own process ID and its parent's process ID.

Procedure

1. Create a Python file: `nano process_id.py`
2. Enter the program given below.
3. Save the file: press **Ctrl + O**, then **Enter**, then **Ctrl + X**.
4. Run the program: `python3 process_id.py`

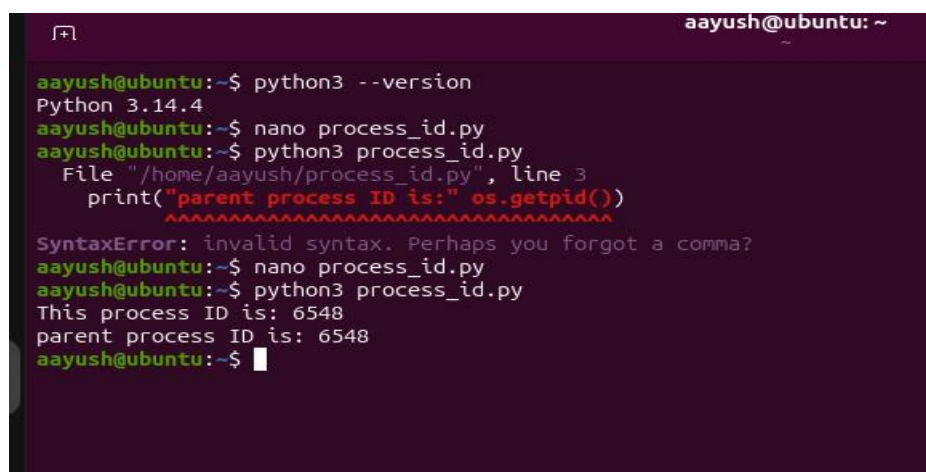
Program

```
import os
print("Process ID:", os.getpid())
print("Parent Process ID:", os.getppid())
```



```
aayush@ubuntu: ~ -- nano process_id.py
GNU nano 8.7.1 process_id.py *
import os
print("This process ID is:", os.getpid())
print("parent process ID is:", os.getpid())
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^G Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Output



```
aayush@ubuntu: ~
aayush@ubuntu:~$ python3 --version
Python 3.14.4
aayush@ubuntu:~$ nano process_id.py
aayush@ubuntu:~$ python3 process_id.py
File "/home/aayush/process_id.py", line 3
    print("parent process ID is:" os.getpid())
                                ^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
aayush@ubuntu:~$ nano process_id.py
aayush@ubuntu:~$ python3 process_id.py
This process ID is: 6548
parent process ID is: 6548
aayush@ubuntu:~$
```

The process IDs will be different on every computer and on every run.

Explanation

Function	Purpose
os.getpid()	Returns the ID of the current process
os.getppid()	Returns the ID of the parent process

Result: The program printed the process ID of the running Python process and the ID of its parent process (the shell).

Experiment 2: Creating a Process Using fork()

Aim

To create a child process using the Linux fork() system call.

Procedure

1. Create the file: `nano fork_demo.py`
2. Write the program given below.
3. Save with **Ctrl + O**, **Enter**, **Ctrl + X**.
4. Run: `python3 fork_demo.py`

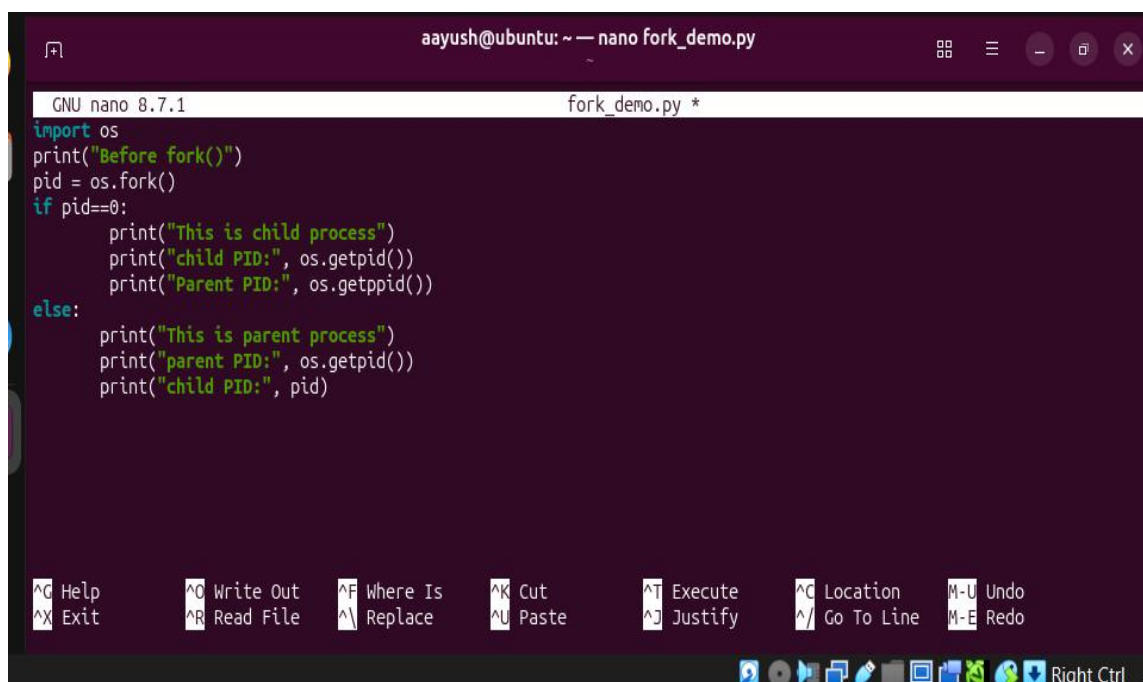
Program

```
import os

print("Before fork()")

pid = os.fork()

if pid == 0:
    print("This is the child process")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
else:
    print("This is the parent process")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)
```



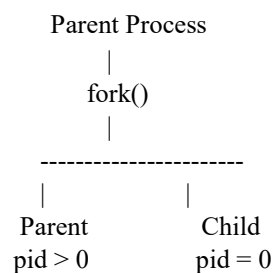
```
aayush@ubuntu: ~ — nano fork_demo.py
GNU nano 8.7.1 fork_demo.py *
import os
print("Before fork()")
pid = os.fork()
if pid==0:
    print("This is child process")
    print("child PID:", os.getpid())
    print("Parent PID:", os.getppid())
else:
    print("This is parent process")
    print("parent PID:", os.getpid())
    print("child PID:", pid)
```

Output

```
aayush@ubuntu:~$ nano fork_demo.py
aayush@ubuntu:~$ python3 fork_demo.py
Before fork()
This is parent process
parent PID: 6892
child PID: 6893
This is child process
child PID: 6893
Parent PID: 6892
aayush@ubuntu:~$
```

"Before fork()" is printed only once, because it runs before the child exists. After fork(), the parent and the child both continue from the same point, so the parent block and the child block are both printed.

Explanation



The value returned by fork() tells each process which one it is:

- **0:** the executing process is the child
- **Positive value:** the executing process is the parent, and the value is the child's PID
- **Negative value:** process creation failed

Result: A child process was created with fork(), and both the parent and the child printed their own process IDs.

Experiment 3: Parent Waiting for Child Using wait()

Aim

To demonstrate process synchronization, where the parent waits for the child process to finish.

Procedure

1. Create the file: nano wait_demo.py
2. Write the program given below and save it.
3. Run: python3 wait_demo.py

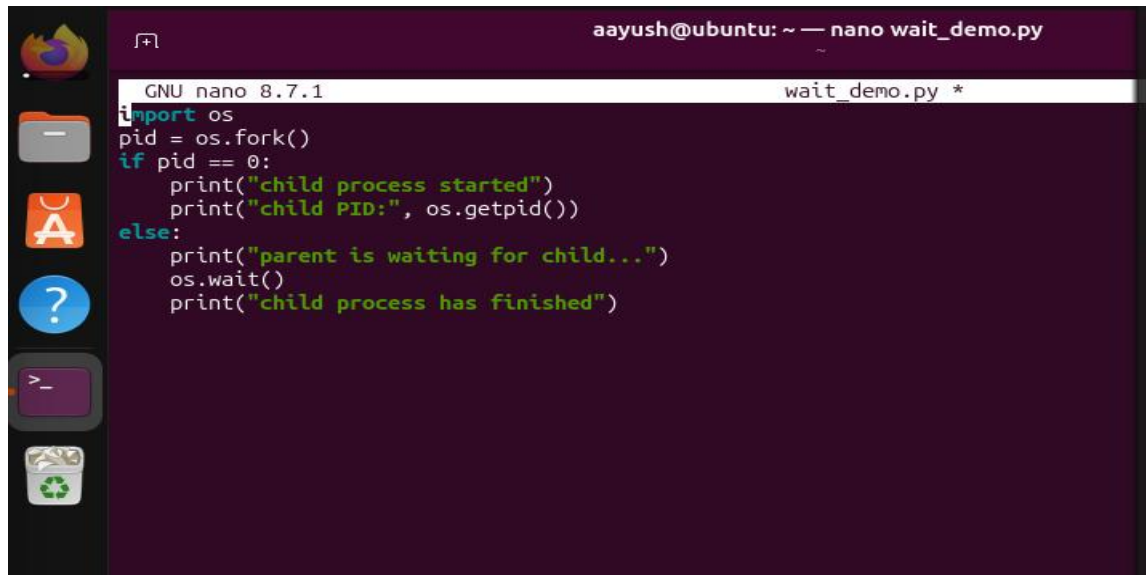
Program

```
import os

pid = os.fork()

if pid == 0:
    print("Child process started")
    print("Child PID:", os.getpid())

else:
    print("Parent is waiting for child...")
    os.wait()
    print("Child process has finished")
```

A screenshot of a terminal window on an Ubuntu system. The window title is 'aayush@ubuntu: ~ — nano wait_demo.py'. The terminal shows the GNU nano 8.7.1 editor with the following Python code:

```
import os
pid = os.fork()
if pid == 0:
    print("child process started")
    print("child PID:", os.getpid())
else:
    print("parent is waiting for child...")
    os.wait()
    print("child process has finished")
```

 The code is displayed with syntax highlighting: keywords in blue, strings in green, and comments in grey. The terminal background is dark purple.

Output



```
aayush@ubuntu: ~  
aayush@ubuntu:~$ nano wait_demo.py  
aayush@ubuntu:~$ python3 wait_demo.py  
parent is waiting for child...  
child process started  
child PID: 5318  
child process has finished  
aayush@ubuntu:~$
```

Explanation

The parent calls `os.wait()`, which makes it wait until the child terminates. Without this synchronization, the parent may finish before the child does. Here the line "Child process has finished" is always printed last, after the child has completed.

Result: The parent process waited for the child process to terminate using `wait()`, and then continued its execution.

Experiment 4: File System Calls

Aim

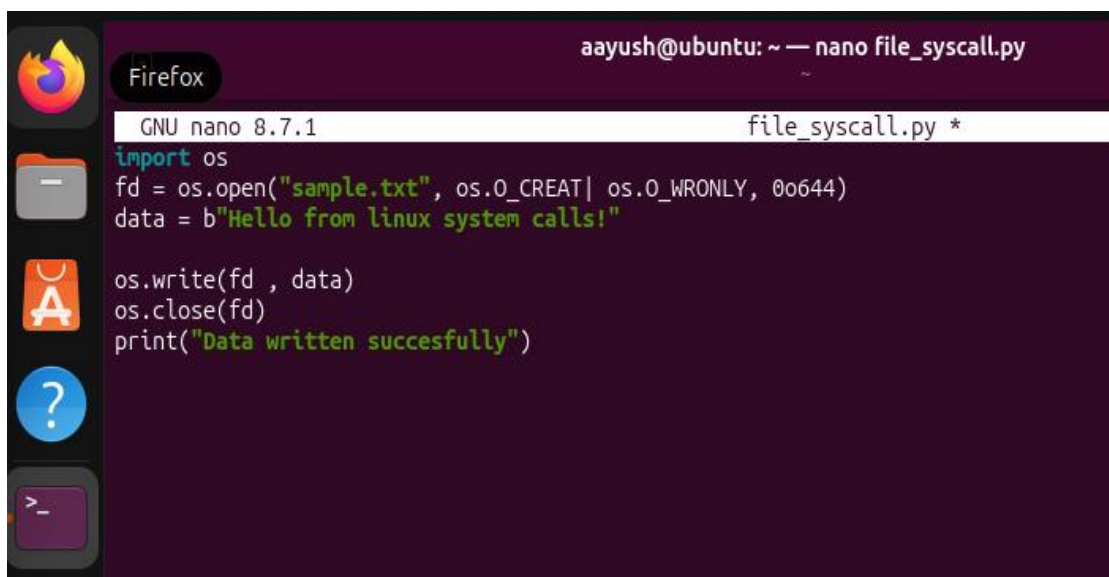
To create a file and write data into it using the Linux system calls `open()`, `write()` and `close()`.

Procedure

1. Create the file: `nano file_syscall.py`
2. Write the program given below and save it.
3. Run: `python3 file_syscall.py`
4. Check that the file exists: `ls`
5. Display its contents: `cat sample.txt`

Program

```
import os
fd = os.open("sample.txt", os.O_CREAT | os.O_WRONLY, 0o644)
data = b"Hello from Linux system calls!"
os.write(fd, data)
os.close(fd)
print("Data written successfully")
```

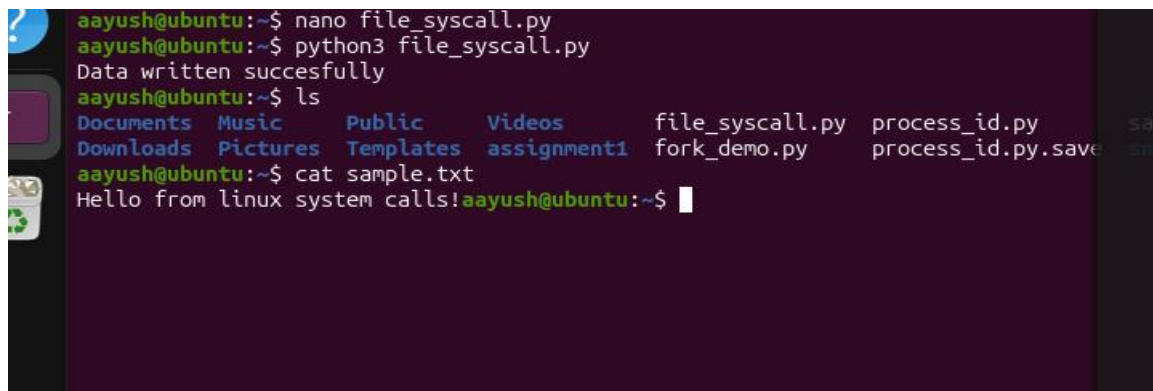


The screenshot shows a terminal window with the title bar "aayush@ubuntu: ~ — nano file_syscall.py". The terminal is running the GNU nano 8.7.1 text editor. The editor's status bar at the top indicates "file_syscall.py *". The code being edited is as follows:

```
import os
fd = os.open("sample.txt", os.O_CREAT | os.O_WRONLY, 0o644)
data = b"Hello from linux system calls!"

os.write(fd , data)
os.close(fd)
print("Data written succesfully")
```

Output



```
aayush@ubuntu:~$ nano file_syscall.py
aayush@ubuntu:~$ python3 file_syscall.py
Data written succesfully
aayush@ubuntu:~$ ls
Documents  Music      Public     Videos    file_syscall.py  process_id.py  process_id.py.save
Downloads  Pictures   Templates  assignment1 fork_demo.py     process_id.py.save
aayush@ubuntu:~$ cat sample.txt
Hello from linux system calls!aayush@ubuntu:~$
```

Explanation

System call	Purpose
os.open()	Opens the file (creates it if it does not exist) and returns a file descriptor
os.write()	Writes the given bytes to the file
os.close()	Closes the file descriptor

Result: The file sample.txt was created and the text "Hello from Linux system calls!" was written into it using file system calls.

Experiment 5: Reading a File Using System Calls

Aim

To read the contents of a file using the system calls `open()`, `read()` and `close()`.

Procedure

1. Create the file: `nano read_file.py`
2. Enter the program given below and save it.
3. Run: `python3 read_file.py`

Program

```
import os
fd = os.open("sample.txt", os.O_RDONLY)
data = os.read(fd, 100)
os.close(fd)
print("File contents:")
print(data.decode())
```

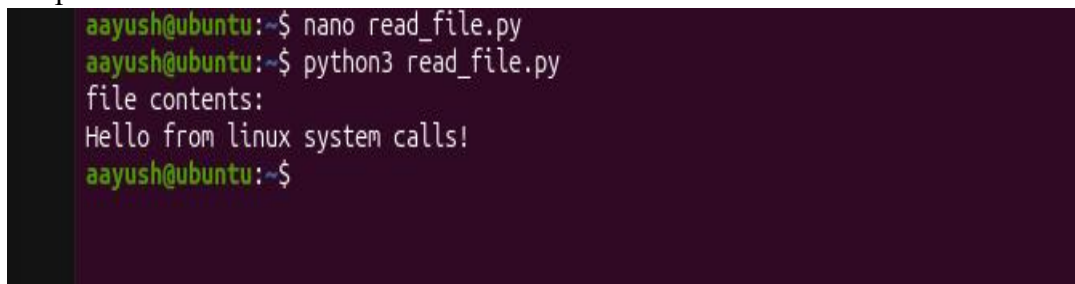


```
GNU nano 8.7.1 read_file.py *
import os
fd = os.open("sample.txt", os.O_RDONLY)
data = os.read(fd, 100)

os.close(fd)

print("file contents:")
print(data.decode())
```

Output



```
aayush@ubuntu:~$ nano read_file.py
aayush@ubuntu:~$ python3 read_file.py
file contents:
Hello from linux system calls!
aayush@ubuntu:~$
```

Explanation

The sequence of calls used in this program is:

```
os.open()  
  |  
  v  
os.read()  
  |  
  v  
os.close()
```

Result: The contents of sample.txt were read using open(), read() and close() and displayed on the screen.

Experiment 6: exec() System Call

Aim

To replace the current process with another program using the exec() family of system calls.

Procedure

1. Create the file: `nano exec_demo.py`
2. Enter the program given below and save it.
3. Run: `python3 exec_demo.py`

Program

```
import os

print("Before exec()")

os.execlp("ls", "ls")

print("After exec()")
```



```
aayush@ubuntu: ~ — nano exec_demo.py
GNU nano 8.7.1 exec_demo.py *
import os
print("before exec()")
os.execlp("ls", "ls")
print("after exec()")
```

Output



```
aayush@ubuntu:~$ nano exec_demo.py
aayush@ubuntu:~$ python3 exec_demo.py
before exec()
Documents Pictures Videos file_syscall.py process_id.py.save snap
Downloads Public assignment1 fork_demo.py read_file.py wait_demo.py
Music Templates exec_demo.py process_id.py sample.txt
aayush@ubuntu:~$
```

Explanation

After "Before exec()" the output of the `ls` command appears. The line `print("After exec()")` is never executed, because `exec()` replaces the current process with the new program.

Python Process

|
exec()

|
v
ls program

The PID stays the same, but the program image running under that PID is replaced.

Result: The Python process was replaced by the ls program using `exec()`, so the statements after `exec()` did not run.

Experiment 7: Complete Process Creation

Aim

To combine the process system calls `fork()`, `getpid()`, `getppid()` and `wait()` in a single program.

Procedure

1. Create the file: `nano process_creation.py`
2. Enter the program given below and save it.
3. Run: `python3 process_creation.py`

Program

```
import os

print("Program started")
print("Original PID:", os.getpid())

pid = os.fork()

if pid == 0:
    print("\n--- Child Process ---")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())

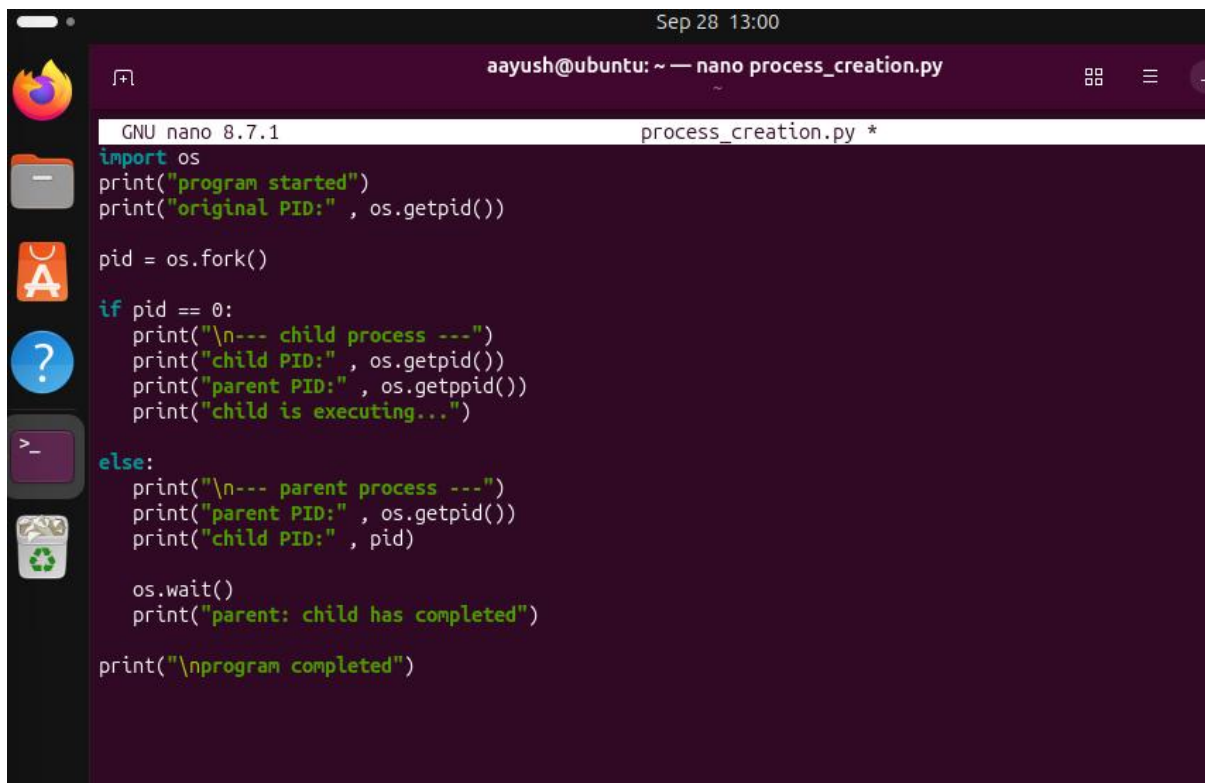
    print("Child is executing...")

else:
    print("\n--- Parent Process ---")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    os.wait()

    print("Parent: Child has completed")

print("\nProgram completed")
```



```
GNU nano 8.7.1 process_creation.py *
import os
print("program started")
print("original PID:" , os.getpid())

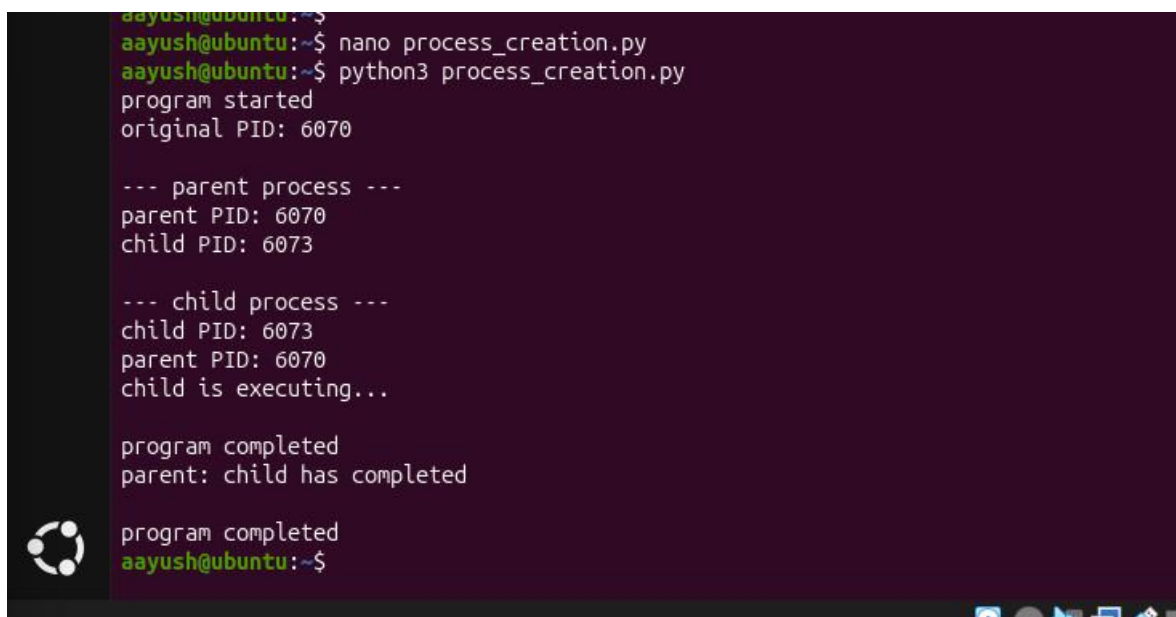
pid = os.fork()

if pid == 0:
    print("\n--- child process ---")
    print("child PID:" , os.getpid())
    print("parent PID:" , os.getppid())
    print("child is executing...")
else:
    print("\n--- parent process ---")
    print("parent PID:" , os.getpid())
    print("child PID:" , pid)

    os.wait()
    print("parent: child has completed")

print("\nprogram completed")
```

Output



```
aayush@ubuntu:~$ nano process_creation.py
aayush@ubuntu:~$ python3 process_creation.py
program started
original PID: 6070

--- parent process ---
parent PID: 6070
child PID: 6073

--- child process ---
child PID: 6073
parent PID: 6070
child is executing...

program completed
parent: child has completed

program completed
aayush@ubuntu:~$
```

Explanation

The parent prints its own PID and the child PID, then waits for the child. The child prints its own PID and its parent PID.

Result: A child process was created with `fork()`, the parent waited for it with `wait()`, and the process IDs were displayed using `getpid()` and `getppid()`.

Important Linux Commands for the Lab

Command	Purpose
pwd	Show current directory
ls	List files
cd	Change directory
mkdir	Create directory
touch	Create file
cat	Display file
nano	Edit file
cp	Copy file
mv	Move/rename file
rm	Delete file
ps	Display processes
clear	Clear terminal
python3 file.py	Execute Python program

Conclusion

In these experiments, the process system calls `getpid()`, `getppid()`, `fork()`, `wait()` and `exec()`, and the file system calls `open()`, `write()`, `read()` and `close()`, were run through Python's `os` module on Linux. The experiments show how a process is created, how a parent synchronizes with its child, how a process image is replaced, and how files are handled at the system call level.